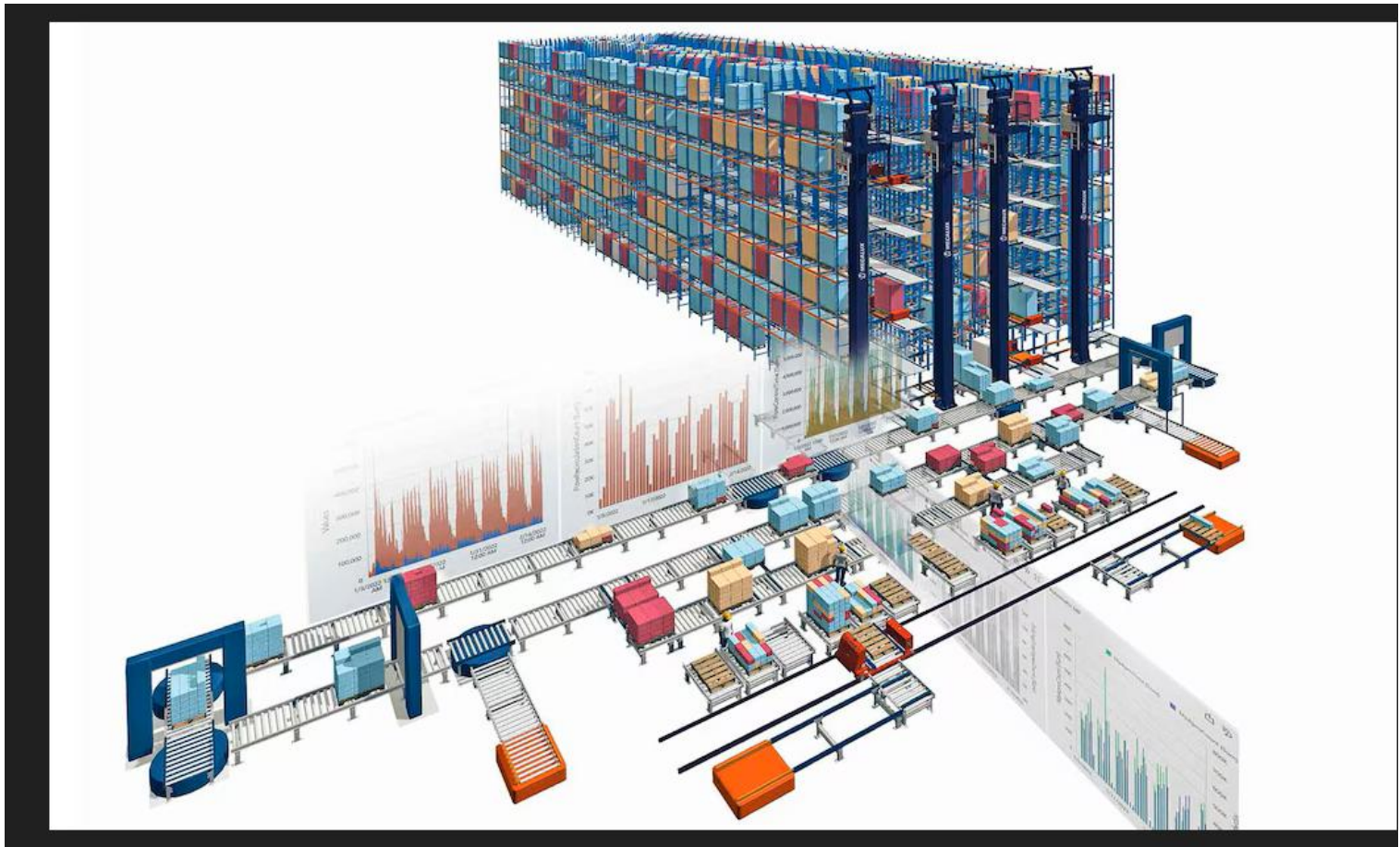


MODULE : ALGORITHMES AVANCES



Dr SADA ANNE

Chapitre 1

Listes

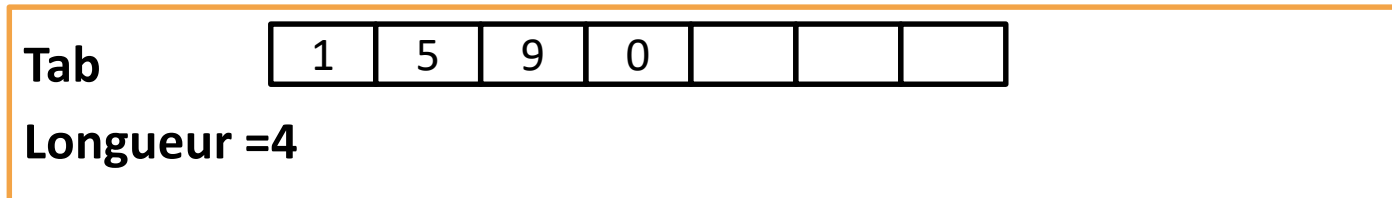
Définition et types

Définition: Une liste est une suite finie (éventuellement vide) d'éléments de même type repérés selon leur rang dans la liste.

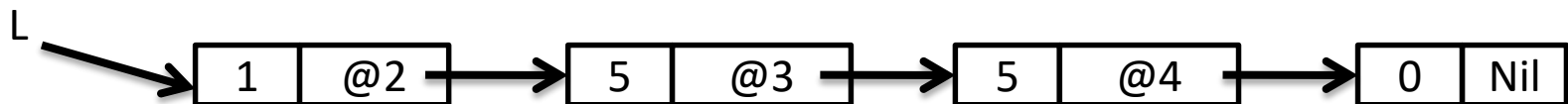
Exemple: la liste d'entiers $L = \{1, 5, 9, 0\}$
le rang de 5 est 2.

Types: deux types

- **Liste statique(contiguë):** Représentation par un tableau d'éléments **Tab** et une variable **longueur** :



- **Liste dynamique:** représentation par des **cellules** ou **maillons**:



Listes statiques

Définition de type liste

Type Liste = Enregistrement

Début

Tab: Tableau[MAX] d'Éléments ;

longueur : Entier ;// garde le nombre d'éléments stockés dans la liste

Fin

Exemple: liste d'entiers

Type ListeEntiers = Enregistrement

Début

Tab: Tableau[1000] d'entiers ;

longueur : Entier;

Fin

Listes statiques

Opérations Quelques opérations sur les listes statiques, on peut ajouter d'autres

- Initialiser(var L:liste)
- Est_vide(L : Liste)
- Est_pleine (L : Liste)
- Longueur (L: liste)
- Ajouter(x : Élément, var L: Liste)
- Premier (l: liste)
- ieme (position: entier, L: liste)
- Supprimer (x: Élément , var L: liste)
- Insérer (x: Élément , pos: entier, var L: liste)

Listes statiques

Procédure Initialiser(var L:liste)

Début

 L.longueur $\leftarrow$ 0;

Fin

Fonction Est_vide(L : Liste) : Booléen

Début

 Retourne (Longueur(l) == 0);

Fin

Procédure Ajouter(x : Élément, var L : Liste)

Début

 L.Tab [L.Longueur+1] $\leftarrow$ x

 L.Longueur $\leftarrow$ L.Longueur+1

Fin

Fonction Longueur (L: liste): entier

Début

 Retourne (L.longueur);

Fin

Fonction Premier (l: liste): Élément

Début

 Retourne (L.Tab [1])

Fin

Fonction ieme (pos: entier, l: liste): Élément

Début

 Retourne (L.Tab [pos])

Fin

Listes statiques

Procédure Supprimer (x : Élément, var L : Liste)

 i, pos : entier

Début

 i=1;

 Tantque i <= **longueur(L)** et T[i] != x faire

 i++;

 Fintantque

 Pos $\leftarrow$ i;

 Si Pos <= longueur(L) alors

 Pour i $\leftarrow$ pos à longueur(L) -1 faire

 L.TabEléments[i] $\leftarrow$ L.TabEléments[i+1]

 Finpour

 L.longueur $\leftarrow$ L.longueur -1;

 Fins

Fin

Listes statiques

Procédure SupprimerPos (pos : entier, var L : Liste)

Var: i :entier

Début

Si (pos <= Longueur (L)) alors

Pour i ← pos à longueur(L) -1 faire

L.TabEléments[i] ← L.TabEléments[i+1]

Finpour

L.longueur ← L.longueur -1;

Finsi

fin

Listes statiques

Avantages

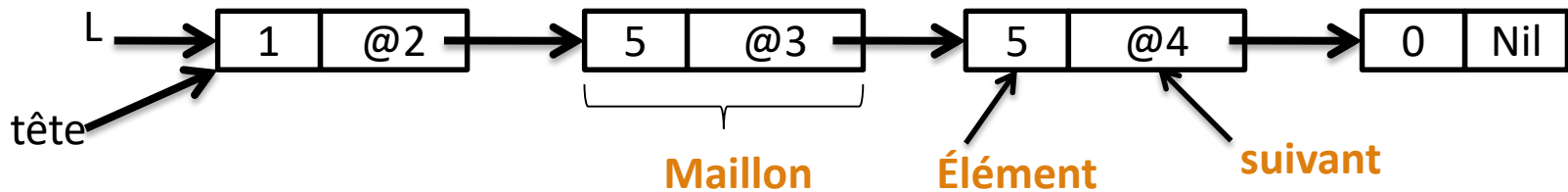
- Parcours et accès faciles au ieme élément (accès direct).
- Possibilité de recherche efficace si la liste est triée (par exemple, recherche dichotomique).

Inconvénients

- Réserve, lors de la compilation, de la Taille maximale imposée (limitée), ce qui manque de souplesse et d'économie.
- Inefficacité de la suppression et de l'insertion à l'intérieur de la liste : **obligation de décaler tous les éléments entre l'élément inséré ou supprimé et le dernier élément.**

Listes chaînées

- Une liste linéaire chaînée (**LLC**) est un ensemble de **maillons**, alloués dynamiquement, chaînés entre eux.



- Un **maillon** est un enregistrement avec deux champs :
 - Champ **Élément** (Ele) contenant l'information.
 - Champ **Suivant** donnant l'adresse du prochain maillon.
- L'adresse de premier élément d'une LLC est souvent appelé tête de la liste.
- **Nil** (**NULL** en **C**) est utilisé pour indiquer la fin de la liste dans le dernier maillon.

Listes chaînées

a) Définition du type liste

Type Structure Maillon

Ele : **typeq**; // **typeq** désigne un type quelconque (int, float, personne, étudiant ...etc.
suivant: *** Maillon**;

fin

Type Liste : *** Maillon**; // Le type liste désigne tout pointeurs vers un maillon

- Utilisation pour la déclaration

L : Liste; ⇔ L: * Maillon;

L, P, Q : Liste // signifie que L, P, Q sont des pointeurs vers des maillons

Listes chaînées

b) Définition de type liste en C++

struct maillon

{

typeq Ele;

 maillons *suivant;

};

typedef maillon *Liste;

Utilisation pour la déclaration

Liste L, Q, **tete**; équivalent à **Maillon** * L, Q, **tete**;

Listes chaînées

Exemple 1: liste chaînée d'entiers

Type Structure Maillon

Ele : entier;

suivant: * Maillon;

Fin

type Liste : * Maillon;

Listes chaînées

Exemple 2: liste chaînée de personnes

Type Structure Personne

nom: chaîne de caractères

prénom: chaîne de caractères

Âge : entier

fin

Type

Structure Maillon

Ele : personne;

suivant: * Maillon;

fin

type Liste : * Maillon;

Listes chaînées

Exemple 2: Liste chaînée d'étudiants

Type Structure étudiant

P: Personne

Numéro: entier

Fin

Type Structure Maillon

Ele: étudiant;

Suivant: * Maillon;

Fin

Type Liste : * Maillon;

Ou

Type Liste_étudiants : *maillon

Listes chaînées (Opérations primitives)

1. **cree_maillon**: crée un nouveau maillon, avec une réservation dynamique de la mémoire (Allouer), contenant comme valeur **x** dans le champs éléments et **Nil** dans le champs suivant. Cette fonction retourne l'adresse du maillon créé.

Fonction creer_maillon (x : typeq): Liste

P: Liste // ou P: *Maillon

Début

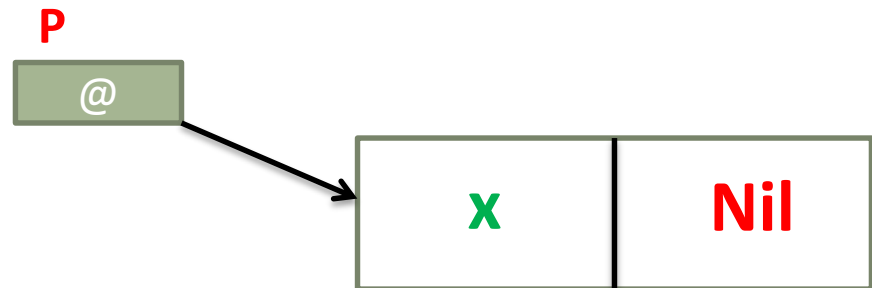
P ← **Allouer** (Maillon)

P -> Éle ← x;

P -> suivant ← Nil;

Retourne (P);

Fi n



Listes chaînées (Opérations primitives)

2. Est_vide: teste si la liste est vide ou non

Fonction Est_vide(L : Liste) : Booléen

Début

Si (L=Nil) alors

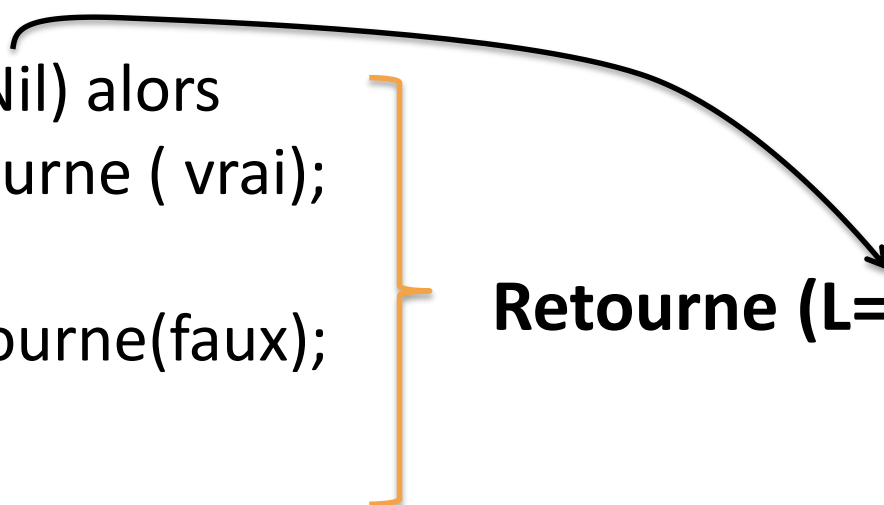
Retourne (vrai);

Sinon

Retourne(faux);

Finsi

Fin



Retourne (L=Nil)

Listes chaînées (Opérations primitives)

3. Premier : retourne le premier élément de la liste L

Fonction Premier (l: liste): Élément,

Début

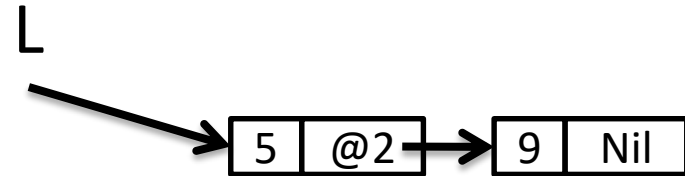
retourne (L -> Ele);

Fin

Exemples d'utilisation

écrire (premier (l));

x ← premier (L);



L -> ELE = 5

Listes chaînées (Opérations primitives)

4. Ajouter: insère un nouveau élément en tête de la liste L. On peut l'écrire comme une fonction, **retournant le nouvelle liste**, ou comme une procédure avec un passage par variable de la liste.

Fonction Ajouter(x : typeq, L: Liste): Liste

P: Liste // ou **P: *Maillon**

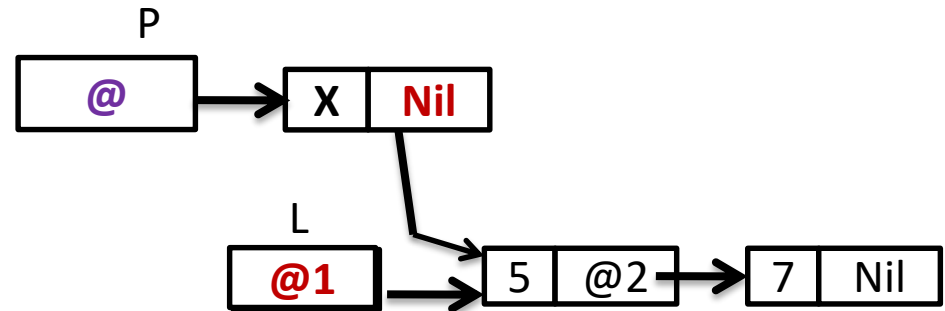
Début

 P ← créer_maillon (x);

 P -> suivant ← L;

 Retourne (P);

Fi n



Listes chaînées (Opérations primitives)

Procédure Ajouter (x : Élément, var L: Liste)

P: Liste

Début

P $\leftarrow$ creer_maillon (x);

P -> suivant $\leftarrow$ L;

L $\leftarrow$ P;

Fi n

Listes chaînées (Opérations primitives)

5. reste: retourne la liste L sans le premier élément. Elle retourne l'adresse du deuxième maillon de la liste (**retourne la valeur du champs suivant du premier maillon**).

Fonction Reste (L: Liste): Liste

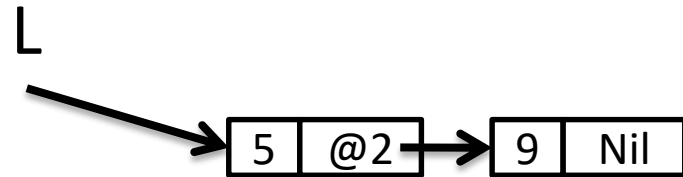
Début

Retourne (L -> suivant);

Fi n

Exemples d'utilisation

écrire (premier (reste (L)));



premier (reste (L))= 9

Listes chaînées (Opération primitives)

6. Longueur: retourne le nombre d'éléments de la liste L.

Version récursive:

Fonction Longueur(L: Liste): entier

Début

Si (L=Nil) alors

Retourne (0);

Sinon

Retourne **1 + longueur(Reste (L));** // appel récursif

Finsi

Fi n

Listes chaînées (Opérations primitives)

6. Longueur: retourne le nombre d'éléments de la liste L.

Version itérative:

Fonction Longueur(L: Liste): entier

 courant : Liste

 Nb: entier

Début

 Nb $\leftarrow$ 0; courant $\leftarrow$ L;

 Tantque **courant $\neq$ Nil** faire

 Nb $\leftarrow$ Nb+1;

 courant $\leftarrow$ courant ->suivant.

 Fintantque

 retourne Nb;

Fi n

Listes chaînées (Autre opérations)

Afficher: permettant d'afficher les éléments de la liste L.

Version récursives

Procédure Afficher (L: Liste) // version récursive.

Début

 Si (L!=Nil)

 écrire (premier (L))

Afficher (reste (L)) // appel récursif

 Finsi

Fi n

Listes chaînées (Autre opérations)

Afficher: permettant d'afficher les éléments de la liste L.

Version itérative

Procédure afficher (L: Liste)

 courant : Liste;

 Nb: entier;

Début

 courant ← L;

 Tantque **courant** != **Nil** faire

 ecrire (**courant** -> **Ele**);

 courant ← courant -> suivant.

 Fintantque

Fi n

Listes chaînées (Autre opérations)

Dernier: retourne le dernier élément de la liste L. La liste doit contenir au moins un élément.

Version récursives

Fonction Dernier (L: Liste): élément

Début

Si (est_vide (reste (L))) alors

retourne premier (L)

Sinon

retourne Dernier (reste (L)) // appel récursif

Finsi

Fi n

Listes chaînées (Autre opérations)

Dernier: retourne le dernier élément de la liste L. La liste doit contenir au moins un élément.

Version itérative

Fonction Dernier (L: Liste): élément

Début

 courant ← L;

 Tantque **courant-> suivant ≠ Nil** faire

 courant ← courant ->suivant.

 Fintantque

 retourne (**courant ->Ele**);

Fi n